

Contents

Euer Bot — Anleitung für Schüler	1
Das Spielprinzip in drei Sätzen	1
Wo schreibt ihr euren Code?	1
Wie eine Entscheidung funktioniert	2
1. Was ihr über die Welt wisst: <i>sensors</i>	2
2. Was ihr tun könnt: <i>Action</i>	4
3. Nützliche Kotlin-Bausteine für eure Bot-Logik	5
4. Ein Bot, Schritt für Schritt aufgebaut	5
5. Bot testen	6
6. Häufige Anfängerfehler	7
Wo ihr weitermachen könnt	7

Euer Bot — Anleitung für Schüler

Diese Anleitung zeigt euch, wie ihr einen eigenen Roboter baut und steuert. Ihr müsst nichts von der Spiel-Engine, der Grafik oder den Regeln selbst programmieren — das ist alles schon fertig. Eure einzige Aufgabe: **eine Funktion namens `decide()` schreiben**, die in jeder Spielrunde einmal aufgerufen wird und sagt, was euer Roboter als Nächstes tut.

Das Spielprinzip in drei Sätzen

Das Spielfeld ist ein 10×10-Raster. Jeder Roboter hat 100 Lebenspunkte (HP) und verliert bei jedem Treffer 10 HP. Das Spiel läuft in **Ticks** (Runden) ab: in jedem Tick darf jeder noch lebende Roboter genau eine Aktion machen — sich bewegen, schießen, oder warten.

Wo schreibt ihr euren Code?

In eurer Team-Datei, z.B. `bots/teamA/TeamABots.kt`. Sie sieht am Anfang so aus:

```
package bots.teamA
```

```
import framework.arena.Action
import framework.arena.Direction
import framework.arena.RobotBrain
import framework.arena.Sensors
```

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        return Action.Move(Direction.SOUTH)
    }
}
```

```
val teamABots: List<RobotBrain> = listOf(MeinBot())
```

Ihr müsst nur den **Inhalt von `decide()`** ändern (die Zeile mit `return Action.Move(...)` und alles, was ihr davor noch braucht). Alles andere in dieser Datei bleibt normalerweise unverändert.

Wichtig, leicht zu vergessen: Legt ihr eine ganz neue Bot-Klasse an, muss sie unten in die Liste `teamABots = listOf(...)` eingetragen werden — sonst taucht sie in der App gar nicht in der Auswahl auf, obwohl der Code fehlerfrei kompiliert.

Wie eine Entscheidung funktioniert

Bei jedem Tick ruft die Engine `decide(sensors)` auf. Ihr bekommt ein `sensors`-Objekt mit allem, was ihr über die aktuelle Situation wissen dürft, und müsst genau eine `Action` zurückgeben. Mehr passiert nicht — kein eigener Loop, kein “warten auf Ereignis”, einfach: rein `Sensors`, raus `Action`.

```
Sensors  ----> decide() ----> Action
(was ich sehe)                (was ich tue)
```

1. Was ihr über die Welt wisst: `sensors`

`sensors` ist euer einziges Fenster zur Welt. Es enthält:

Feld	Bedeutung
<code>sensors.self</code>	euer eigener Roboter-Zustand
<code>sensors.others</code>	Liste aller noch lebenden gegnerischen Roboter
<code>sensors.arenaWidth / sensors.arenaHeight</code>	Größe des Spielfelds (10 × 10)
<code>sensors.tick</code>	die aktuelle Runden-Nummer

Tote Gegner tauchen in `sensors.others` nicht mehr auf — ihr müsst also nie selbst prüfen, ob ein Gegner schon besiegt ist.

Euer eigener Zustand: `sensors.self`

```
sensors.self.position // eure Position, z.B. Position(x=3, y=7)
sensors.self.health   // eure aktuellen HP, z.B. 80
sensors.self.id       // eure interne ID, z.B. "bot-0"
```

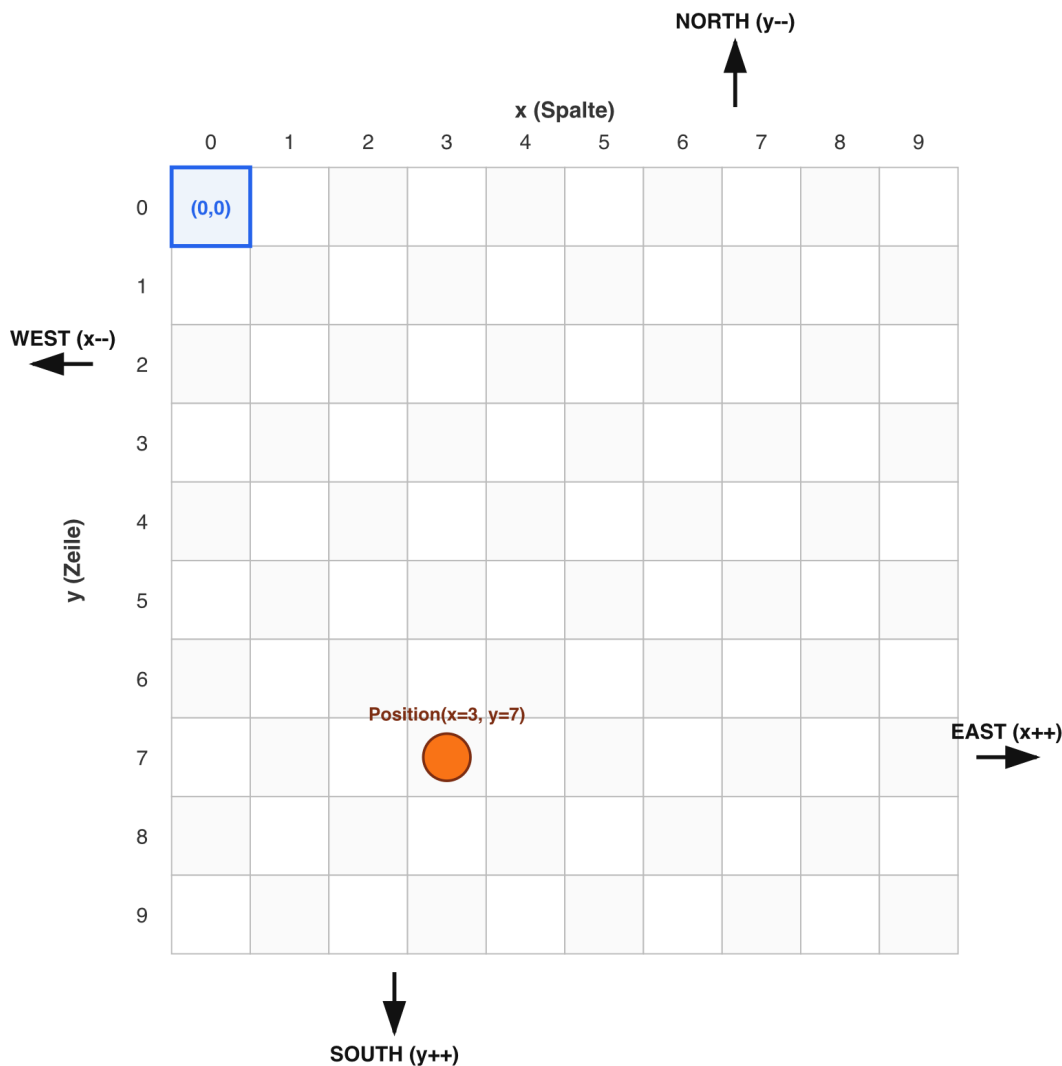
Beispiel — eure eigene Gesundheit abfragen:

```
if (sensors.self.health < 20) {
    // wenig HP übrig – hier könnte Flucht-Logik stehen
}
```

Positionen: `Position(x, y)`

Jede Position hat ein `x` (Spalte) und ein `y` (Zeile). `(0, 0)` ist **oben links**.

Achtung, wichtige Stolperfalle: `y` wird **kleiner**, je weiter oben man ist, und **größer**, je weiter unten. Nach Norden (oben) laufen heißt also: `y` sinkt, nicht steigt.



```

val meinePosition = sensors.self.position
val binAmLinkenRand = meinePosition.x == 0
val binAmOberenRand = meinePosition.y == 0

```

Gegner: `sensors.others`

Eine Liste, jeder Eintrag hat dieselben Felder wie `sensors.self` (`.position`, `.health`, `.id`).

Beispiel — gibt es überhaupt noch einen Gegner?

```

val gegner = sensors.others.firstOrNull()
if (gegner == null) {
    // keiner mehr da, z.B. gewonnen
}

```

Für Fragen wie “wer ist der nächste Gegner?” oder “wer hat am wenigsten HP?” müsst ihr nicht selbst rechnen — dafür gibt es fertige Toolkit-Funktionen:

```

val naechsterGegner = sensors.nearestEnemy() // null, wenn keiner mehr lebt
val schwachsterGegner = sensors.weakestEnemy() // null, wenn keiner mehr lebt

```

Volle Übersicht aller Helferfunktionen (Distanz, Richtung, Sichtlinie, Rand/Mitte, ...): [toolkit-referenz.md](#).

2. Was ihr tun könnt: Action

Jede `decide()`-Funktion muss **genau eine** von drei Aktionen zurückgeben:

```
Action.Move(Direction.NORTH)    // einen Schritt in eine Richtung gehen
Action.Shoot(Direction.EAST)     // in eine Richtung schießen
Action.Wait                      // nichts tun
```

Die vier möglichen Richtungen: `Direction.NORTH`, `Direction.SOUTH`, `Direction.EAST`, `Direction.WEST` (Norden, Süden, Osten, Westen).

Achtung, Schreibweise-Falle: `Action.Wait` schreibt man **ohne** Klammern. `Action.Move(...)` und `Action.Shoot(...)` brauchen **mit** Klammern eine Richtung als Parameter. Ohne Klammern bei `Move/Shoot`, oder mit Klammern bei `Wait`, gibt es einen Compile-Fehler.

Wichtig zu wissen: Euer Bot schlägt mit einer Action nur vor, was passieren soll — die Engine setzt das nach ihren eigenen Regeln um. Wenn ihr z.B. `Action.Move(Direction.NORTH)` zurückgebt, aber dort steht schon ein anderer Roboter oder das Feld liegt außerhalb der Arena, passiert einfach nichts. Es gibt keine Fehlermeldung — der Zug verpufft, und im nächsten Tick dürft ihr wieder entscheiden.

Schießen trifft nur in gerader Linie: Ein Schuss nach EAST trifft nur einen Gegner, der in **exakt derselben Zeile** (gleiches y) rechts von euch steht. Steht kein Gegner in dieser Linie, verpufft der Schuss wirkungslos — es passiert nichts Schlimmes, aber ihr richtet auch keinen Schaden an.

Ein Roboter blockiert die Sichtlinie. Trifft euer Schuss nicht den Gegner, den ihr im Sinn hattet, sondern steht ein anderer Roboter näher in derselben Linie, wird stattdessen der getroffen. Ihr könnt also nicht “durch” einen Roboter hindurchschießen.

Ihr könnt euch nicht durch andere Roboter hindurchbewegen. Ein Zielfeld, auf dem zu Beginn des Ticks noch ein lebender Roboter steht, ist blockiert — auch wenn dieser Roboter im selben Tick selbst wegzieht. Alle Bewegungswünsche eines Ticks werden nämlich gleichzeitig gegen den Zustand vor dem Tick geprüft, nicht nacheinander. Daraus folgen drei Fälle:

- **Besetztes Feld:** Ziel war zu Tick-Beginn belegt -> ihr bleibt stehen, auch wenn der Bewohner im gleichen Tick wegzieht.
- **Kein Platztausch:** Zwei Roboter, die genau die Plätze tauschen wollen, bleiben beide stehen (ist quasi eine Sonderform des besetzten Felds).
- **Zielkonflikt:** Wollen zwei oder mehr Roboter ins selbe freie Feld, bewegt sich keiner von ihnen.

Reihenfolge in einem Tick: erst bewegen, dann schießen. Die Engine löst in jedem Tick zuerst *alle* Bewegungen auf und danach *alle* Schüsse. Das hat eine wichtige Folge für Treffer: Ob ein Schuss trifft, wird an der Position des Ziels **nach** dessen Bewegung geprüft, nicht vorher.

- Ein Gegner, der sich in diesem Tick aus eurer Zeile/Spalte **heraus** bewegt, wird **nicht** getroffen — obwohl er zu Tick-Beginn noch in der Linie stand.
- Ein Gegner, der sich in diesem Tick **in** eure Zeile/Spalte **hinein** bewegt (z.B. euch gerade kreuzt), **kann** getroffen werden — obwohl er vorher nicht in Linie war.

Ihr seht die Zukunft nicht. `sensors.others` zeigt euch die Positionen der Gegner zu *Beginn* des Ticks — also **bevor** sie sich bewegen. Wenn ihr entscheidet, wisst ihr noch nicht, wohin ein Gegner in diesem Tick zieht. Ein Schuss auf ein bewegliches Ziel ist deshalb immer eine **Vorhersage**, kein sicherer Treffer. Zuverlässiger trifft ihr Gegner, deren nächsten Zug ihr abschätzen könnt — z.B. einen, der stillsteht, oder indem ihr euch so positioniert, dass ihr eine wahrscheinliche Route abdeckt.

3. Nützliche Kotlin-Bausteine für eure Bot-Logik

Kurzreferenz für Dinge, die in den Beispielen oben vorkommen und die ihr für eure eigene Logik wiederverwenden könnt:

Baustein	Beispiel	Bedeutung
<code>?:</code> (Elvis-Operator)	<code>sensors.others.firstOrNull { ?: return Action.Wait</code>	Wenn links null rausbekommt, dann mach stattdessen das rechts vom <code>?:</code>
<code>.firstOrNull()</code>	<code>sensors.others.firstOrNull {</code>	erstes Element einer Liste, oder null falls leer
<code>.minByOrNull { }</code>	<code>sensors.others.minByOrNull { it.health }</code>	Element mit dem kleinsten Wert (z.B. wenigste HP)
<code>.random()</code>	<code>Direction.entries.random()</code>	zufälliges Element aus einer Liste/Aufzählung
if/else als Ausdruck	<code>val x = if (a) 1 else 2</code>	if/else kann direkt einen Wert liefern, nicht nur verzweigen
when	siehe <code>ChaserBot.kt</code> in <code>bots/examples/</code>	wie eine große, übersichtliche if/else if-Kette

4. Ein Bot, Schritt für Schritt aufgebaut

Die Bausteine von oben ergeben zusammengesetzt einen immer fähigeren Bot — hier als Ausbaustufen, jede baut auf der vorherigen auf.

Stufe 0 — nichts tun

```
override fun decide(sensors: Sensors): Action {  
    return Action.Wait  
}
```

Der einfachste mögliche Bot. Macht nichts, verliert aber auch nie durch eigene dumme Züge.

Stufe 1 — zufällig bewegen

```
override fun decide(sensors: Sensors): Action {  
    val richtung = Direction.entries.random()  
    return Action.Move(richtung)  
}
```

`Direction.entries` ist die Liste aller vier Richtungen, `.random()` wählt eine zufällig aus. Das entspricht der ersten Backlog-Story (1.1).

Stufe 2 — immer in eine feste Richtung schießen

```
override fun decide(sensors: Sensors): Action {  
    return Action.Shoot(Direction.EAST)  
}
```

Einfach, aber trifft nur Gegner, die zufällig genau östlich von euch in derselben Zeile stehen.

Stufe 3 — auf einen Gegner in Sichtlinie zielen

Für “steht er in meiner Zeile/Spalte?” und “in welche Richtung schießen?” müsst ihr nicht selbst mit x/y vergleichen — dafür gibt es passende Toolkit-Funktionen (volle Übersicht: [toolkit-referenz.md](#)):

```
override fun decide(sensors: Sensors): Action {
    val gegner = sensors.nearestEnemy() ?: return Action.Wait

    return if (sensors.canShoot(gegner)) {
        // gleiche Zeile oder Spalte – canShoot() prüft das für uns
        Action.Shoot(sensors.self.position.directionTo(gegner.position)!!)
    } else {
        Action.Move(Direction.SOUTH) // sonst irgendwie bewegen
    }
}
```

`sensors.canShoot(gegner)` prüft, ob der Gegner in derselben Zeile oder Spalte steht, und `directionTo` liefert die passende Schussrichtung dazu — deshalb ist das `!!` hier sicher, `directionTo` gibt nur bei nicht ausgerichteten Positionen null zurück. Sonst bewegt sich der Bot (hier fest nach Süden — das ließe sich noch verbessern, z.B. mit `approachDirectionTo` in Richtung des Gegners laufen statt fest nach Süden).

Stufe 4 — bei niedriger Gesundheit fliehen

```
override fun decide(sensors: Sensors): Action {
    val gegner = sensors.nearestEnemy() ?: return Action.Wait
    val self = sensors.self.position

    if (sensors.self.health < 20) {
        // fliehen: in die Richtung weg vom Gegner laufen
        return Action.Move(self.fleeDirectionFrom(gegner.position)!!)
    }

    // sonst normal angreifen wie in Stufe 3
    return if (sensors.canShoot(gegner)) {
        Action.Shoot(self.directionTo(gegner.position)!!)
    } else {
        Action.Move(Direction.SOUTH)
    }
}
```

Fragt zuerst die eigene Gesundheit ab (`sensors.self.health < 20`) und verhält sich dann komplett anders. `fleeDirectionFrom` übernimmt dabei das Vorzeichen-Dreh-Problem (weg statt hin) komplett — ihr müsst weder `abs()` noch Vergleiche zwischen x/y-Werten selbst schreiben. Das ist das Grundmuster für Story 1.3 und für Story 3.1 (Zustandsmaschine — euer Bot verhält sich unterschiedlich, je nachdem “in welchem Zustand” er gerade ist, ähnlich einer Ampel).

5. Bot testen

```
./gradlew run
```

startet die App. Wählt in der Bot-Auswahl euren eigenen Bot und einen Beispiel-Bot aus `bots/examples`, startet das Match, und schaut im Scoreboard/Log rechts, was passiert.

Wenn euer Bot einen Fehler hat (z.B. Endlosschleife oder Absturz): keine Sorge, das Spiel stürzt deswegen nicht ab. Euer Bot macht in dem Fall einfach `Wait`, bis ihr den Fehler behoben und neu gestartet habt.

6. Häufige Anfängerfehler

- **Neue Bot-Klasse angelegt, aber nicht in `teamXBots` eingetragen** -> Bot compiliert, taucht aber nicht in der App-Auswahl auf.
- **`Action.Wait` mit Klammern geschrieben** (`Action.Wait()`) -> Compile-Fehler.
- **Richtung verwechselt** — denkt daran: NORTH heißt y wird kleiner (nach oben), SOUTH heißt y wird größer (nach unten).
- **`sensors.others` als leer angenommen, aber nicht geprüft** — wenn ihr `sensors.others.first()` statt `sensors.others.firstOrNull()` benutzt und die Liste ist leer, gibt es eine Exception. Lieber immer `firstOrNull() + ?:-` Fallback benutzen (siehe Beispiele oben).

Wo ihr weitermachen könnt

Das Backlog in [docs/backlog.md](#) enthält alle Aufgaben (User Storys) mit genauen Anforderungen, sortiert nach Schwierigkeit. Fangt mit Epic 1 an.

Wenn ihr bei einer Story nicht weiterkommt: in [docs/story-hinweise.md](#) gibt es zu jeder Story Denkhilfen, Taktik-Gedanken und Leitfragen (mit Bildern) — ohne die Lösung zu verraten.